The Publishing Accelerator Toolkit (PacTk)

Amy Neustein, Ph.D.
CEO and Founder
Linguistic Technology Systems
Fort Lee, NJ
917-817-2184
amy.neustein@verizon.net

Class Libraries and Full-Text Search/Encoding Tools for Publishing Documents, Datasets, and Multimedia Resources

Overview ▸▸▸

In contemporary publishing, the standard workflow includes XML-based formats such as JATS or BITS, from which are compiled PDFs and other reader-visible documents. Intermediate XML and similar files are distinct from those that authors submit directly; they have their own tool stacks and are computationally accessed via protocols such as SAX (Simple API for XML), XQuery, and XQSE (XQuery scripting extension). Unfortunately, XML itself is not explicitly built for representing textual data, and examining JATS or BITS files via generic XML technology renders certain publishing-related query and editing tasks difficult to implement. The PDF format is similarly opaque because internal PDF representations of document text include many anomalies due to ligatures, hyphenation, font specs, and other layout-related discrepancies between XML inputs and user-visible output. As a result, internal PDF text only partially corresponds to textual content as understood by authors and editors.

Given the limitations of PDF and XML for natural-language documents, numerous competing formats have been developed, providing potential alternatives or extensions to XML. These include TEI (Text Encoding Initiative), TagML (Text-as-Graph Markup Language), BioC (a biomedical text mining standard defined by PubMed Central), and Research Object Bundles (a Semantic-Web based specification that covers both text and data sets). With a proliferation of different text-encoding standards, query and traversal protocols that are developed for one format (e.g., JATS and its derivatives) cannot be directly applied to others. For example, the XML parent-child element relationship is replaced in TagML with two different containment styles (logical and extensional), such that DOM and event-based parsers targeted at the simpler XML hierarchy must be reimplemented for the TagML system.

Meanwhile, with regard to PDF, effective search and text-interaction capabilities depend on supplementing this format's internal natural-language representation with machine-readable text encoding, which might be embedded in PDF files or available as sibling assets. In principle, XML documents could supply such content, although current pipelines are not set up to share JATS/BITS files (for example) in end-user contexts.

From the perspective of PacTk, text encoding evinces a spectrum of distinct *topomorphic regimes* where a proliferation of incompatible document structures prohibits the emergence of common query, traversal, and metadata formats. The obvious problem is that workflows have to be reconstructed for each incompatible document specification. In practice, such difficulties have almost certainly hindered the adoption of theoretically well-founded formats such as TagML.

The idea behind PacTk is that divergent formats cannot be satisfactorily reconciled simply by proposing yet another format to insert into analogous workflows. Instead, building a more sophisticated publishing platform involves constructing integrated computational tableaux weaving together text-encoding, query-evaluation, compiler-theoretic, and front-end concerns.

One important variation amongst text-encoding formats is the level of granularity for textual elements. In JATS, BITS, and most JSON encodings, the fundamental discursive unit is the individual paragraph. Elements on smaller scales (such as sentences or block quotes) are notated haphazardly at best. LaTeX employs reasonable sentence-boundary heuristics — which may be overridden via macros when necessary — but sentence objects are not expressed in any systematic manner (e.g., via auxiliary files) for any purposes other than visual spacing. In TEI and BioC, individual sentences *can* be represented, but this is not mandatory (thus **s** and **sentence** tags are available but not guaranteed to be present in any particular document). By contrast, PacTk is explicitly built around sentence-level units, specifically a "topomorphic sentence-level object model" (TSOM). When all regular text in a natural-language document is explicitly contextualized in a delineated sentence object, many editing, indexing, and user-interface tasks become feasible (as outlined in the next section).

In contrast to the other formats mentioned here, PacTk is not a single file type but rather a collection of code libraries and classes, with data types representing specific textual elements (sentences, paragraphs, PDF pages, annotations and comment-boxes, etc.). Query and traversal protocols modeled via XQuery, DOM, and so on become expressed by object methods and operators exposed to C++ code, for

example (PacTk is thus "topomorphic" in the sense that navigation between objects is determined by structural properties of divergent document formats, that manifest in competing notions of individuation for structure-sites and any interrelationships declared between them). In short, the PacTk object system synthesizes distinct structuring protocols found in competing formats such as JATS, TagML, and BioC, allowing publishers' workflows to encompass each of these formats (and others) without sacrificing compatibility. PacTk is particularly aimed at publishers who feel constrained by the limitations of JATS or BITS but are reluctant to try alternatives (TagML, BioC, etc.) due to minimal software-ecosystem support and the lack of standardized XQuery and DOM-style interfaces.

The remainder of this document will discuss how PacTk can streamline editing, indexing, and document-preparation tasks, as well as improve User Experience for human readers. Later sections will also address how PacTk may contribute to broader projects where text documents are published alongside data sets and/or multimedia presentations. Overall, PacTk presents a unified model and coding interface for text queries operating in diverse contexts, including PDF viewers, search engines, and document-preparation code. Similarly, software employing PacTk to navigate through document objects can leverage interop metadata across multiple formats — such as XML elements and PDF page coordinates — that is typically unavailable in other publishing tools.

Features for Editing, Indexing, and PDF Viewers ▸▸▸

The computational foundation of PacTk is a collection of classes that represented individual sentences and other textual elements in the TSOM family (PDF pages, annotations, footnotes, block quotes, edits or changes across document versions, etc.). In order to initialize these objects, PacTk provides a distinct input format called GTagML, or "grounded" TagML. The term "grounded" reflects how certain structure-sites may be specifically marked as serializing a typed object: i.e., tags, attributes, or text nodes in the neighborhood of a site supply the information necessary to initialize an object-instance during deserialization. In other respects GTagML follows most of the principles advocated by TagML, with novel extensions; for example, GTagML encourages the exploitation of Unicode Private Use Planes to disambiguate visibly similar but logically distinct characters/glyphs (e.g., periods may embody punctuation, acronyms, or abbreviations).

In source files, GTagML is loosely based on Markdown, so authors familiar with Markdown from more informal contexts (e.g., social media, chat rooms, GIT documentation, etc.) will recognize many of the GTagML styling commands. GTagML sources might be provided by authors directly, or derived from author inputs by digital copy editors or others on the manuscript pipeline. To support GTagML, PacTk provides a new C++ parsing library that is readily extensible (more so than parsers for XML, JSON, and so forth), allowing publishers to customize the GTagML format with new tags, syntax, and data structures specific to their in-house styles and disciplinary focus areas (e.g., specialized tags/data for different scientific fields — chemistry, physics, medicine, etc.; or whatever subject areas are addressed by given publishers, journals, and book series). Alternatively, TSOM objects could be compiled from sources such as JATS or BITS directly.

For workflows that feature JATS, BITS, or LaTeX, TSOM objects may be employed as adjunct data sources through which edits, queries, and other computational artifacts are routed. PacTk data can then be mapped both to archiving formats (like XML) and camera-ready generators (like LaTeX), but with a common TSOM background these two different representational facets may be integrated (for example, XML elements mapped to corresponding PDF page coordinates). In effect, PacTk serves to triangulate human-visible presentations (like PDF) with behind-the-scenes JATS-style formats. Analyses that are currently implemented against XML structures may be more efficiently coded via PacTk objects, with concrete benefits in areas such as copy-editing and indexing.

▷ PacTk for Editing

PacTk has an advanced query framework to help find specific locations in text where edits might be warranted (e.g., to comport with journal or book-series conventions) and to correctly review and process such edits. Here are some features:

Contextual Filters for Text Locations Often it is useful to restrict the scope of text searches to content with specific presentation characteristics (italics, alternative typefaces, citations), discursive roles (inline/block quotes, footnote text, section/subsection titles), or semantic significance (acronyms, annotations, proper/place names).

Implementing such filters via XPath/tag restrictions can be inaccurate, because relevant filter criteria could be orthogonal to DOM hierarchies. For example, XML or LaTeX code might wish to wrap elements such as footnotes or block quotes in custom environments, to ensure desired pre- or post-processing. Material that is functionally footnote or quoted text, therefore, may not necessarily be indicated with those specific tag names. The PacTk system allows programmers to define lists of boolean parameters that apply to text-sequences and then activate those flags automatically in specific tag contexts.

In general, PacTk supports queries where ordinary textual restrictions may be augmented with filters for a wide variety of layout and discursive contexts. Such queries could then be used for editing tasks that should be localized to specific contexts, or where a user wants to find a specific piece of text known to have a given context.

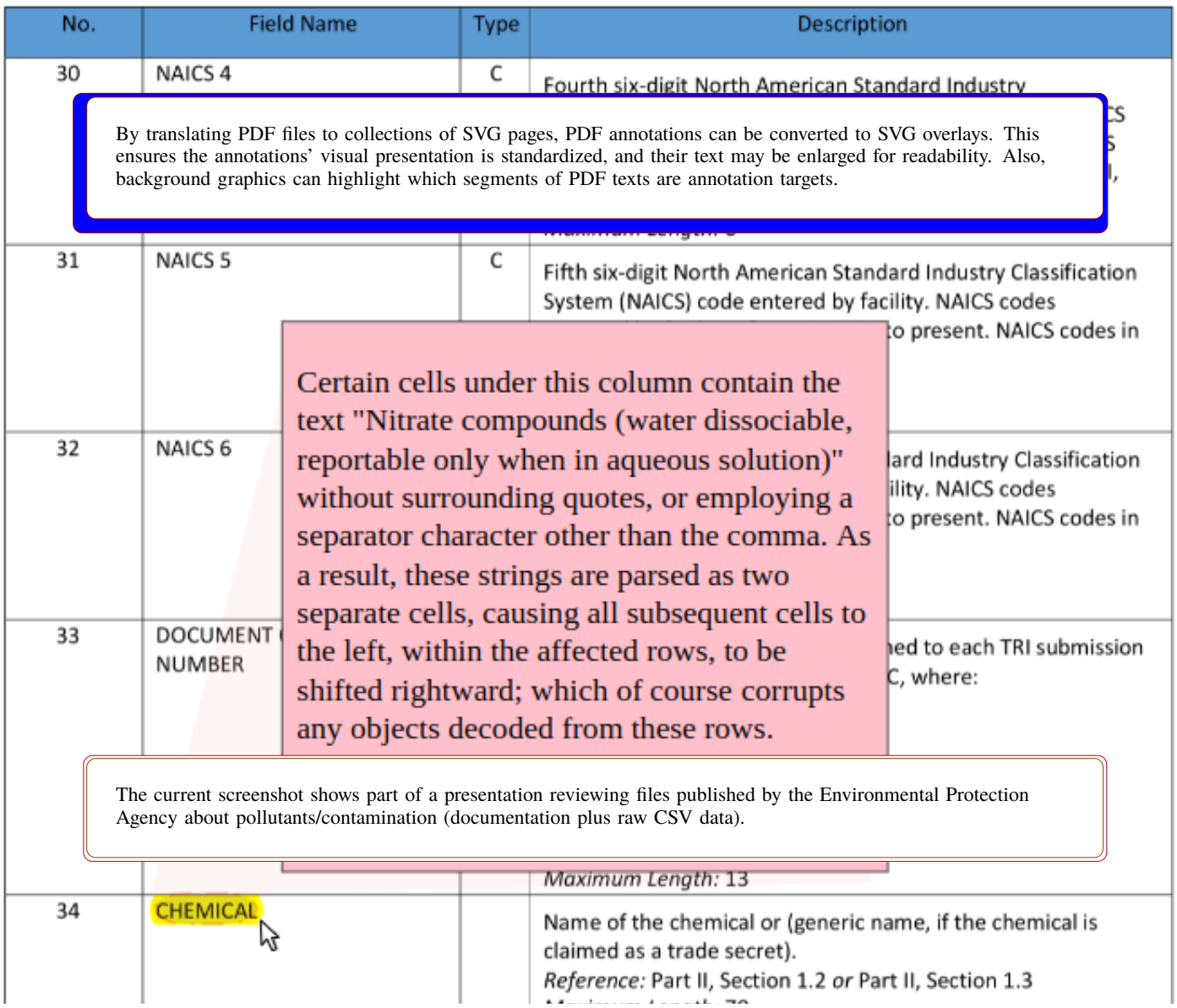


Figure 1: Mapping PDF pages to SVG views for improved annotation support

Sentence-Level Queries Siting queries relative to sentence boundaries can improve precision and express query rationales more succinctly (when compared to paragraph-based organization). For instance, the proper punctuation around a word or phrase often depends on whether that content appears at the beginning, middle, or end of a sentence. Some query expressions might similarly depend on stipulating that two or more search targets (e.g., two proper names) occur in the same sentence.

Because PacTk recognizes almost all text as contained inside sentence objects, queries dependent on sentence boundaries can be directly evaluated on TSOM data (such boundaries are unambiguously notated via input sources, rather than detected approximatively using Natural Language Processing). PacTk likewise models "topomorphic" navigational relations between sentences, such as between adjacent siblings of parent objects (typically paragraphs) and also nestings wherein a multi-sentence footnote, for example, occurs in a parent-sentence context, or where a block quotes functions in part as a logical child of the sentence that introduces it.

PDF Page Coordinates Human readers post-publication — plus authors and copy editors beforehand — typically view documents in PDF format for normal reading, even if edits/changes are made on intermediary files. As a result, a bitmapped page-image is the primary medium through which users view single publication pages. Such graphics are typically rendered by PDF viewers, but other techniques might be used, such as converting page images to SVG (illustrated here in Figure 1, where the relevant component is an embedded web engine that communicates with a PDF viewer by **QWebChannel**; so SVG events are processed by C++ code directly, with only a minimal JavaScript routing layer). Failure to interoperate these images with structured text formats is a significant hindrance to efficient document preparation and editing.

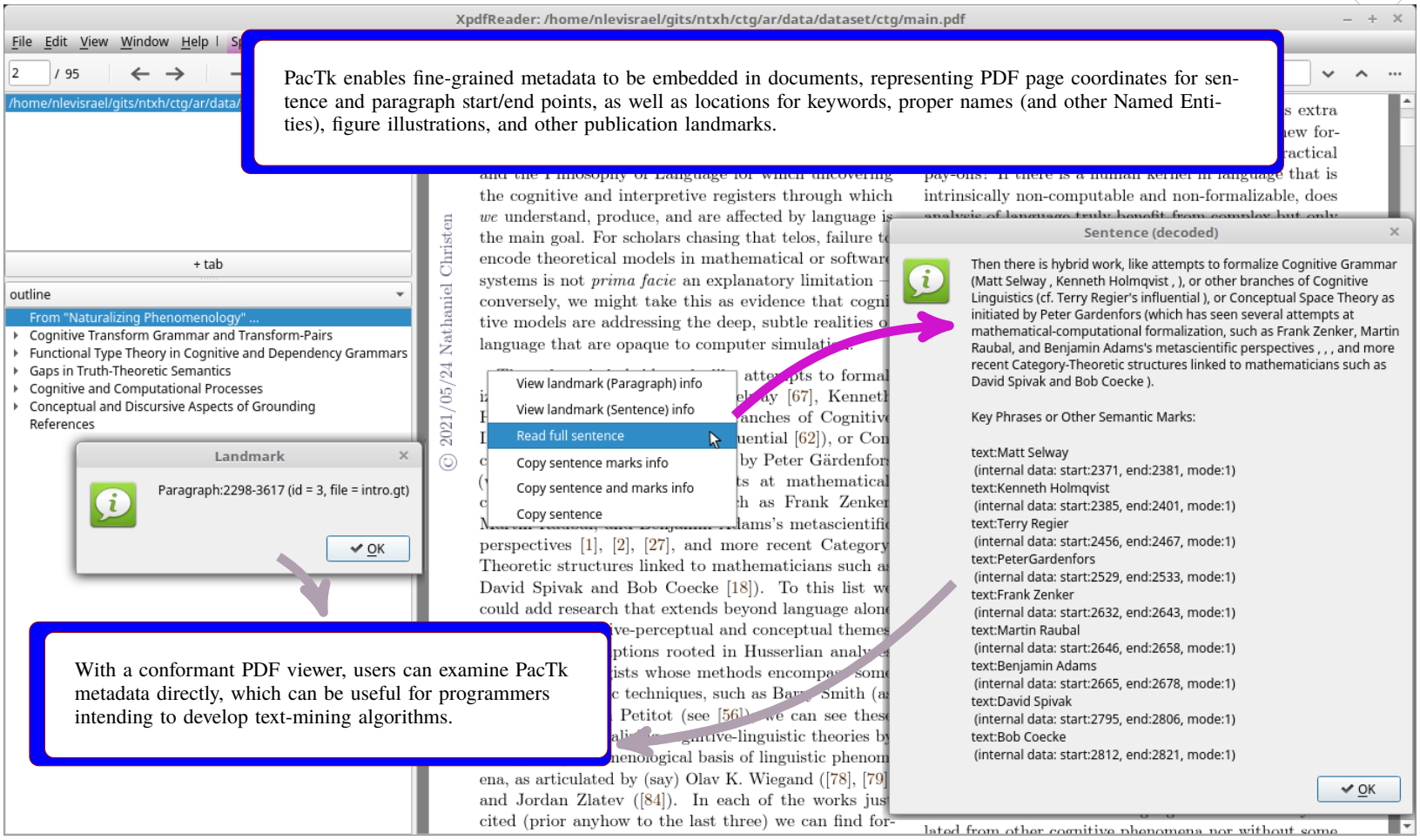


Figure 2: Enhanced GUI capabilities enabled by machine-readable text encoding

By design, **PacTk** supports post-processing algorithms within pipelines whose formats can generate **PDF**-coordinate metadata, such as LaTeX **pgfsavepos** (and auxiliary **write**) commands. Page coordinates for significant discursive locations (e.g., sentence and paragraph boundaries, footnote markers, keyphrases and named entities, citations) can thereby be exposed to **PDF** viewers which could accordingly implement novel front-end features such as overlays, context menus, and dialog windows positioned nearby relevant content (Figure 2). For example, a context menu action may include copying the full text of a current sentence to clipboard, based on the screen position where the menu is activated; semitransparent overlays could similarly show the extent of individual sentences during mouseover events.

PDF Comments and Annotations

In addition to sentence objects, **PacTk** classes model data specifically relevant to the **PDF** front-end milieu, including single pages, links/anchors, and comments/annotations. Objects such as comment-boxes may thereby be leveraged systematically in an editing/review pipeline.

In particular, **PacTk** supports construction of hashtag- or handle-style controlled vocabularies that could annotate **PDF** comments introduced by copy editors or typesetters. Many modifications considered during the editing process follow common patterns, such as proper use of commas around subordinate clauses, or correct formatting of ellipses and bracketed content within inline/block quotes. Assuming that editors consistently include notations for recurring patterns, **PacTk** code can filter queries specifically to **PDF** comment boxes which indicate a specific editing category. Index codes within **PDF** comments may likewise indicate which individual sentence surrounds the annotation, so that the **PDF** view could be cross-referenced against input sources, which facilitates review and processing of manuscript changes.

For similar reasons, **PacTk** supports abstractions such as **XML** custom character entities and LaTeX macros. Many familiar textual components — ellipses, *m*- and *n*-dashes, quotation and footnote marks, etc. — are typically represented not via their visible characters but rather through standardized codes or glyphs. The problem is that systematic mappings are lacking when mixing different file types (e.g., LaTeX and **XML**, such as **JATS** or **BITS**) in a single workflow. **PacTk** has a flexible abstraction system that allows character and macro codes to propagate across different generated files.

A related problem is that abstraction codes should generally be opaque to human users in contexts such as comment boxes and copy-paste between software applications. **PacTk** therefore supports declarations for how custom entities/macros and Unicode Private Use Area codepoints should be rendered in plain-text contexts as well as structured formats such as **XML** and LaTeX.

Often users may prefer to initiate or review edits through **PDF** files directly, but in other cases plain-text files might be appropriate instead, without **GUI** overhead. With **PacTk**, for example, authors could receive a pair of text files (or collections of files indexed, for instance, by chapter) summarizing changes between two manuscript versions. Via sentence-level filtering, such files could be organized such that only

sentences that differ from one version to the next are included, with each sentence listed separately. This may be a more convenient setup for authors to review changes than working through typical GUIs. Of course, such breakdown is only possible when the text encoding systematically identifies individual sentences, as with PacTk's TSOM object model.

▷ PacTk for Indexing and Search Engines

Authors might use several methods to compile an index. They can start with a list of important headings (names, concepts, etc.) and search for relevant passages in the manuscript. Alternatively, authors could read through a document and identify when a particular paragraph mentions a named entity suitable for indexing. Depending on representations or tag sets used (e.g., the JATS **string-name** element) annotated spans and citations may also form the provisional heading-list for an index.

Given these various index-preparation methods, it is useful to extend front-end tools with capabilities specific to indexing workflows, such as query matching and manipulating index entry objects. Structured formats like JATS do not provide sufficient computational tractability in themselves. For example, the most popular desktop GUI framework are the Qt C++ libraries, which are the basis of cross-platform PDF viewers such as Poppler and XPDF. PacTk can seamlessly interoperate with Qt, exposing sentence texts as **QString** objects and expressing GTagML parsing grammars as **QRegularExpression** instances, paired with lambda callbacks activated by the first matching rule (relative to the current parse-position). More precisely, GTagML has context-sensitive grammars that can include or skip over rule-tests via activated contexts; this allows sections of a GTagML file to utilize syntax different from the default.

With respect to indexing, GTagML has its own notation for index entries (headings, locators, and "see also" style redirections), to serialize indexes in a human-readable format. Correspondingly, PacTk has classes specifically for index entries and associated data. These objects could be initialized from GTagML files or specialized windows floating atop PDF viewers.

Building indexes on the basis of native sentence and index-entry objects has certain benefits as compared to generic (e.g., XML) manuscript formats. These include:

Separating Data from Presentation Different publishers or book series may choose different visual styles for their indexes. Representing indexes behind the scenes via structured data objects allows indexes to be curated in abstraction from the specific layout chosen for final publication.

PDF Context Menus Authors/editors may prefer to add new index entries or locators directly from PDF pages. PacTk-compliant viewers will read metadata that specifies which paragraphs are in scope at different page-coordinate regions, so paragraph ids for any location can be ascertained from mouse events (e.g., the dropdown point of a context menu).

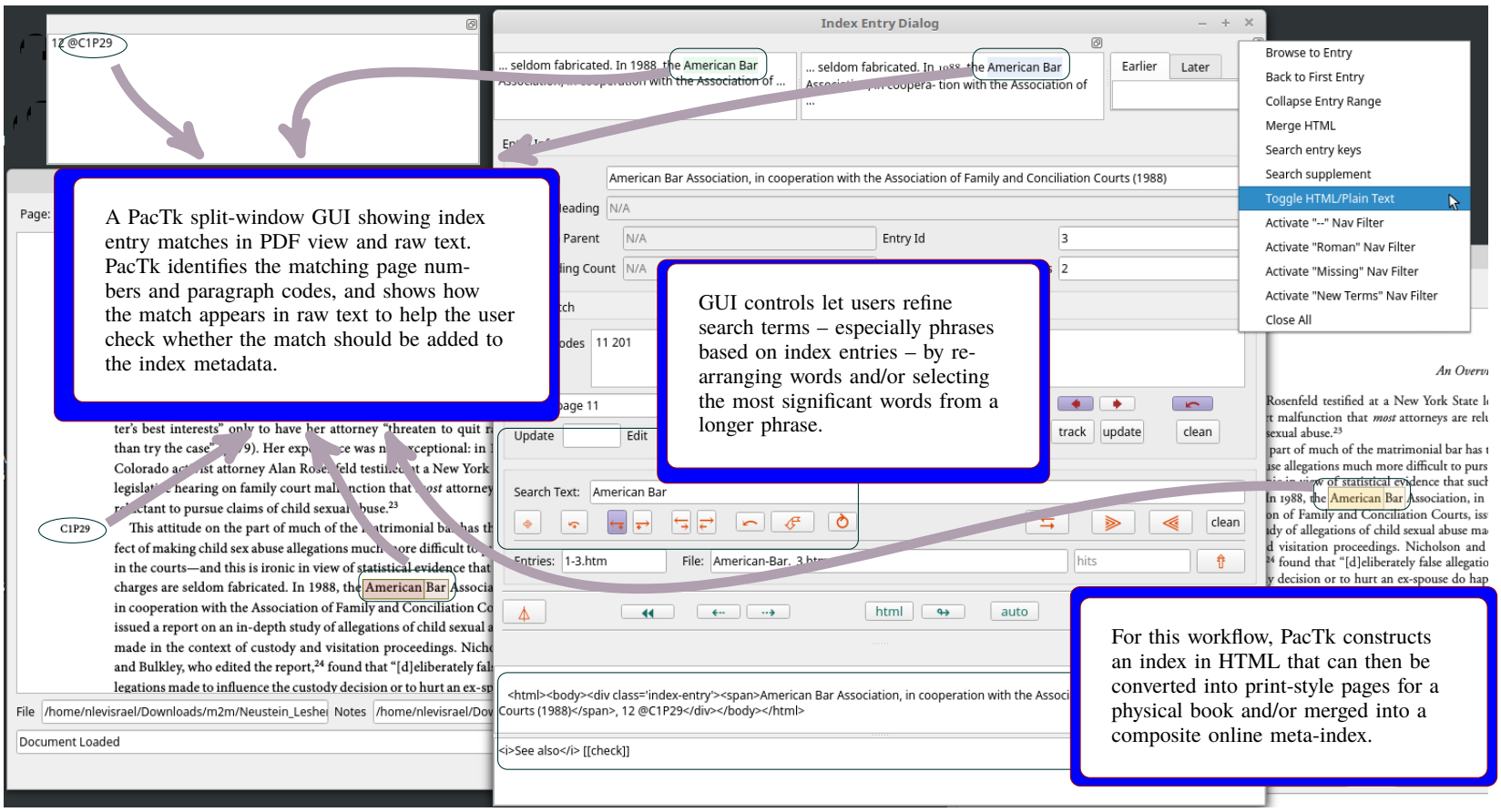


Figure 3: GUI functionality for compiling and curating indexes

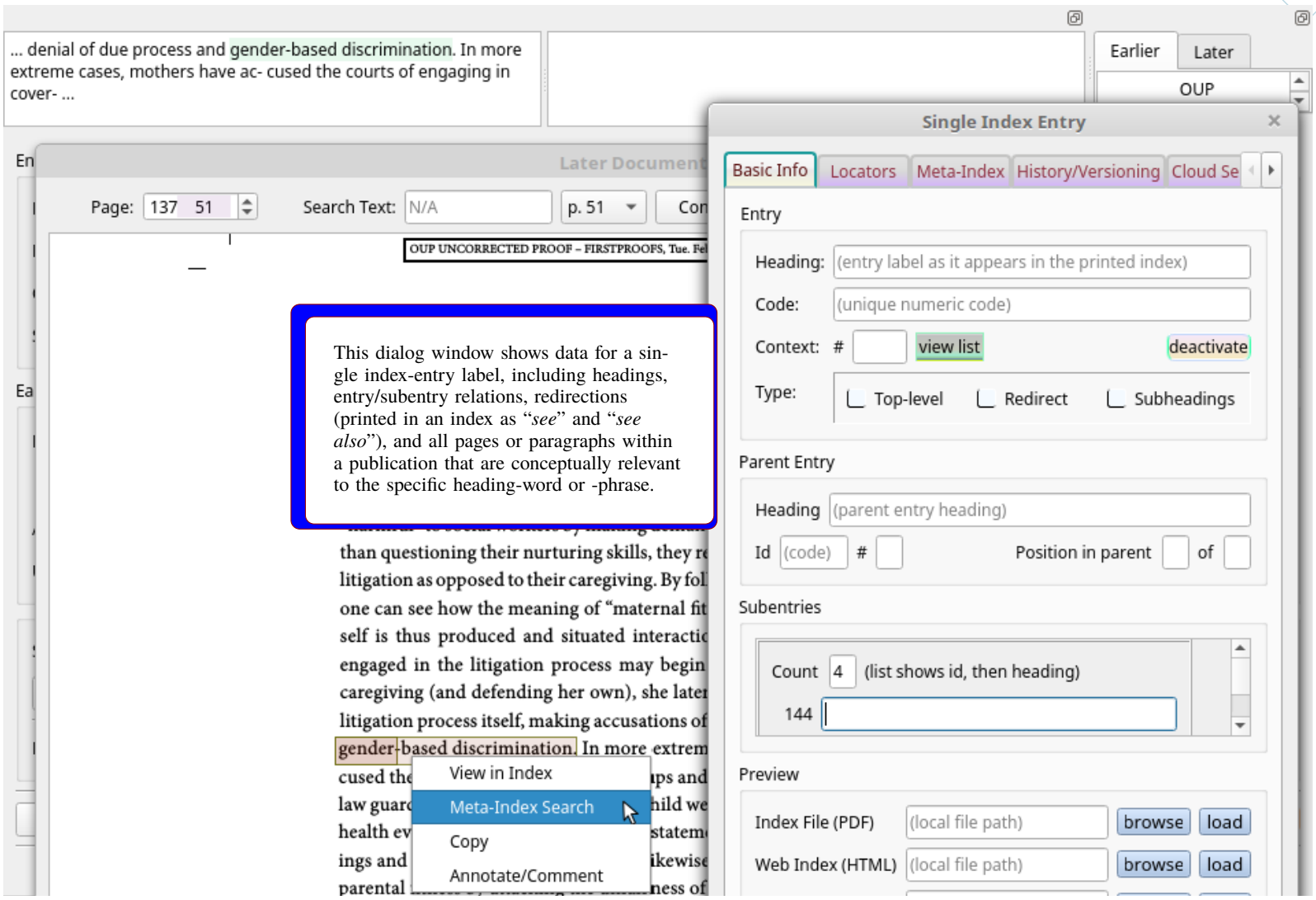


Figure 4: An index entry dialog window (and context menu for single and meta-index views)

Multiple Match Strategies Index locators are typically identified by matching text against entry-headings, but the parameters of these searches will vary by context. Sometimes locators are only relevant when the local sentence includes just the exact heading label, but in other places a desired match could include words in different order (e.g., either first-name/last-name or vice-versa), or a subset (e.g., last name only), or associated concepts with different language entirely (e.g., "The White House" in lieu of a president's name). In the presence of a particular index entry, front-end displays can give users options to fine-tune which match strategies are appropriate for query strings seeking locators specific to that entry.

Index-Building GUI Components Index entries are multi-faceted objects that naturally lend themselves to dedicated front-end displays, such as dialog windows (Figures 3 and 4). For example, entries typically include a heading label and then one or more locators (e.g., paragraph ids, replaced eventually with page numbers), then possible subheadings with their own locators, plus "redirectors" (*see* or *see also*) applied either to overall entries or to subheadings. This represents a lot of information to keep track of for each entry, and indexes can include hundreds of entries. Although indexes are eventually shown to readers as ordinary PDF pages (or similar formats), during document prep it is useful to subdivide indexes into individual GUI windows for each entry, with options to navigate between adjacent entries or to traverse on larger scales (e.g., one chapter to another), plus to create new entries on the fly.

Index-Guided Text Searching Indexes are curated lists of headings and locators selected by authors/editors for relevance. Whenever a search is posed against a manuscript in any context — e.g., in the query bar of a PDF viewer, or an online search engine, or XQuery-style terms evaluated vis-à-vis JATS files — index metadata can be consulted just in case search terms overlap with headings already included in an index. Such functionality may be augmented by extending index metadata to include concepts related to index entries even if those secondary concepts are not indexed themselves. In effect, index metadata may declare that all locators modeled as relevant for their specific entry are also relevant to some list of associated concepts, such that whenever those latter concepts are targeted in a query the explicit locators are included among the query results.

Series or Archival Meta-Indexes Once a standard format is developed for indexes in some document collection, it becomes possible to pool multiple indexes into a common resource. Each time a new publication is finalized, all of its index terms could be inserted into a bibliographic database allowing users to search across multiple books/manuscripts in a given book series, journal collection, or document repository. Such meta-indexes can support searches that are more precise and granular than conventional search-engine technologies based on occurrence vectors and stemming/lemmatization alone (Figure 5).

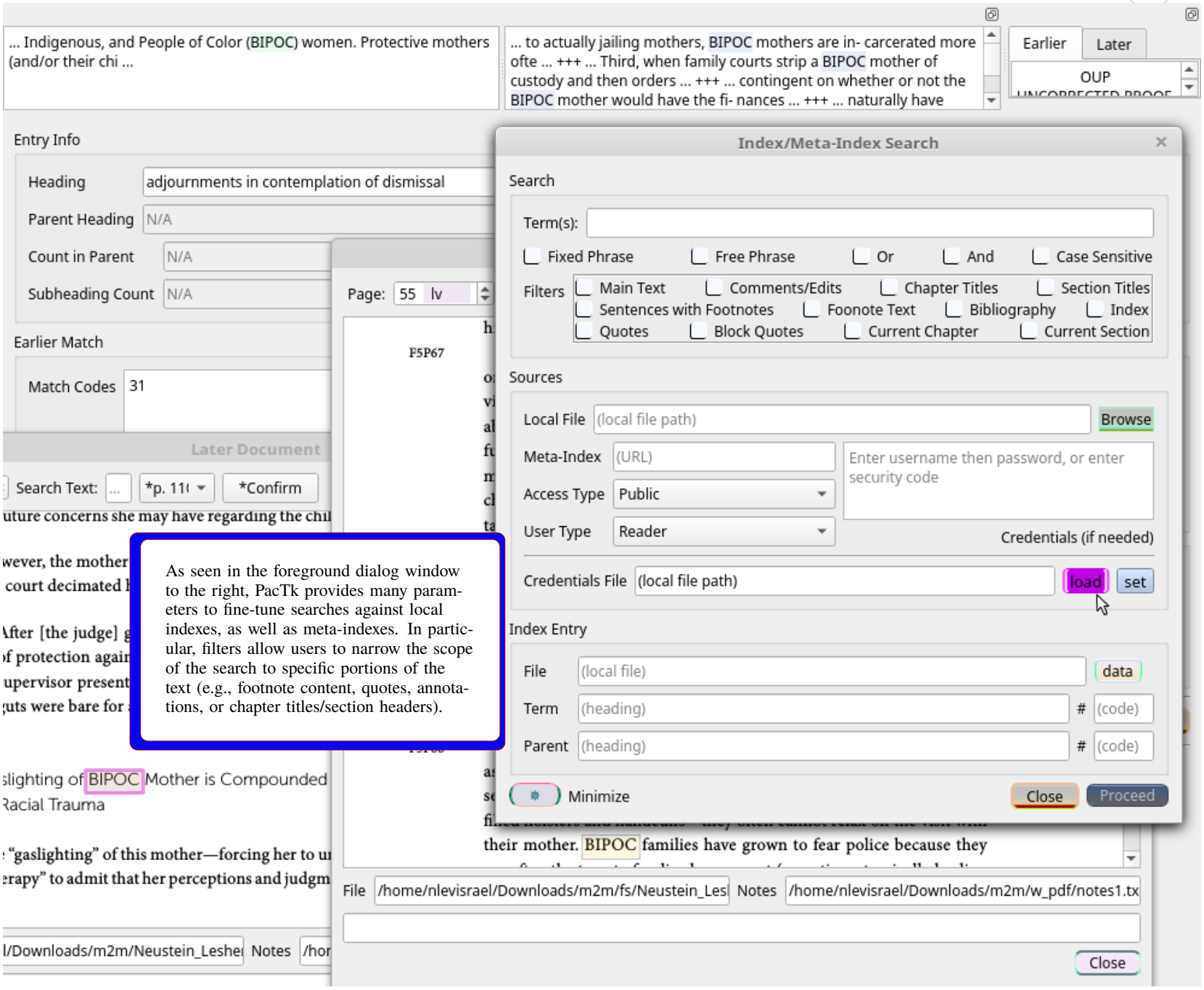


Figure 5: Dialog window for local and meta-index searches

Academic Search Engines In online contexts, "indexing" typically refers not to human-visible book indexes but rather to compressed data files that catalog nontrivial words and concepts in a manuscript. Such indexing is typically non-contextual; for instance, it is not represented whether a word/phrase is present in text with a special purpose (quotes, footnotes, chapter/section titles, etc.) and sentiment/discursive roles are ignored (e.g., a document may be identified as *about* some topic, but more specific criteria such as arguing *for*, *against*, or neutrally *evaluating* theories or claims are not modeled). Granularity may be limited by the scope of an archive, because efficient queries can only expend a few seconds per document if searches are to be conducted against a large collection.

Nonetheless, search functionality can take steps to increase query specificity via meta-indexes and related metadata. For one thing, users may specifically consult search areas that are restricted in scope — say, a single University Press, book series, repository (PubMed Central, for example), or journal collection. Publishers who curate meta-indexes for mid-size collections can support much more detailed evaluations per document; a book series might have several dozen titles, not tens of thousands. PacTk supports construction of corpora query languages that express granular searches over multiple documents, with similar logic to filtered/contextualized text searches in a single PDF document (Figures 6 and 7). Even where queries are executed via conventional vector-database methods, PacTk allows authors or digital editors to exercise control over how word-occurrence lists are compiled. For example, TSOM objects can include lemmatized word-lists per sentence; authors may then control for circumstances such as two different meanings reduced to the same linguistic stem, which might not be properly ingested by search engines indexing documents through their default algorithms.

Although PacTk's originating focus is publishing — where the primary components of a repository are manuscripts/documents in formats like PDF — this technology could potentially be used in other environments. For instance, in clinical trials and point-of-care software the fundamental assets are Electronic Health Records; but search features, as well as plugins and multimedia (discussed next), might be extended to include resources such as diagnostic images, cytometry gating, assay graphics, tumorogenesis simulations, etc.

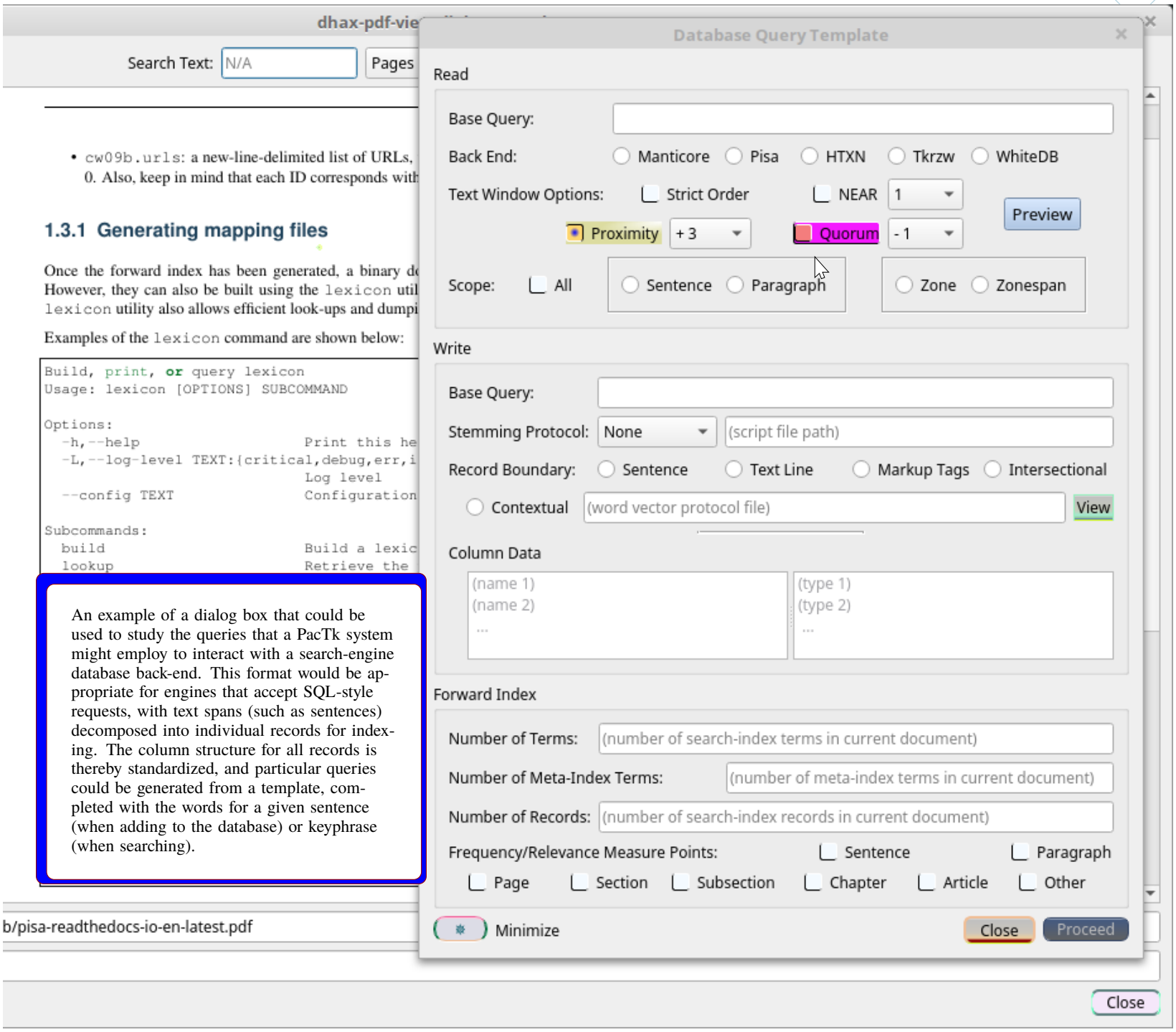


Figure 6: Configuring default back-end queries

Data Publishing and Multimedia ▸▸▸

Contemporary data transparency and FAIR sharing (Findable, Accessible, Interoperable, Reusable) standards introduce new challenges for publishers. In this environment, published documents such as PDFs serve as only one part of a package of files, alongside research data sets and, often, numerous forms of multimedia content (potentially including audio, video, image series, 3D graphics, and statistical/data-visualization tools and diagrams, plus domain-specific interactive formats). If desired, PDF viewers could themselves be implemented as one portion of a multimedia suite, with functional motifs duplicated across the various components (Figures 8 and 9).

As a "publishing accelerator" toolkit, PacTk has several components to help publishers construct these supplemental materials, including cross-referencing text documents with associated data/media assets, and ensuring FAIR usability. Here are several components under development:

Deserialization and Dataset Tools Individual scientific disciplines are often associated with domain-specific data publishing formats that are paired with dedicated deserialization tools (i.e., computer code which reads raw data files into live memory, without relying on special-purpose software). Such code libraries are often maintained by individual institutions or organizations — which might be academic departments, research labs, or independent groups — on behalf of a broader research community. Examples include (among many others) FASTQ and VCF for genomics; HDF5 and ROOT for physics; FITS and NetCDF in astronomy and earth sciences; PDB and MOL in biochemistry; IFC and BIM for architecture/engineering; SEGY and BAG for geophysics and oceanography; CoNLL-U in computational linguistics; DICOM and OMETIFF for bioimaging; and GIS-indexed attribute layers and shapefiles for geospatial data sets.

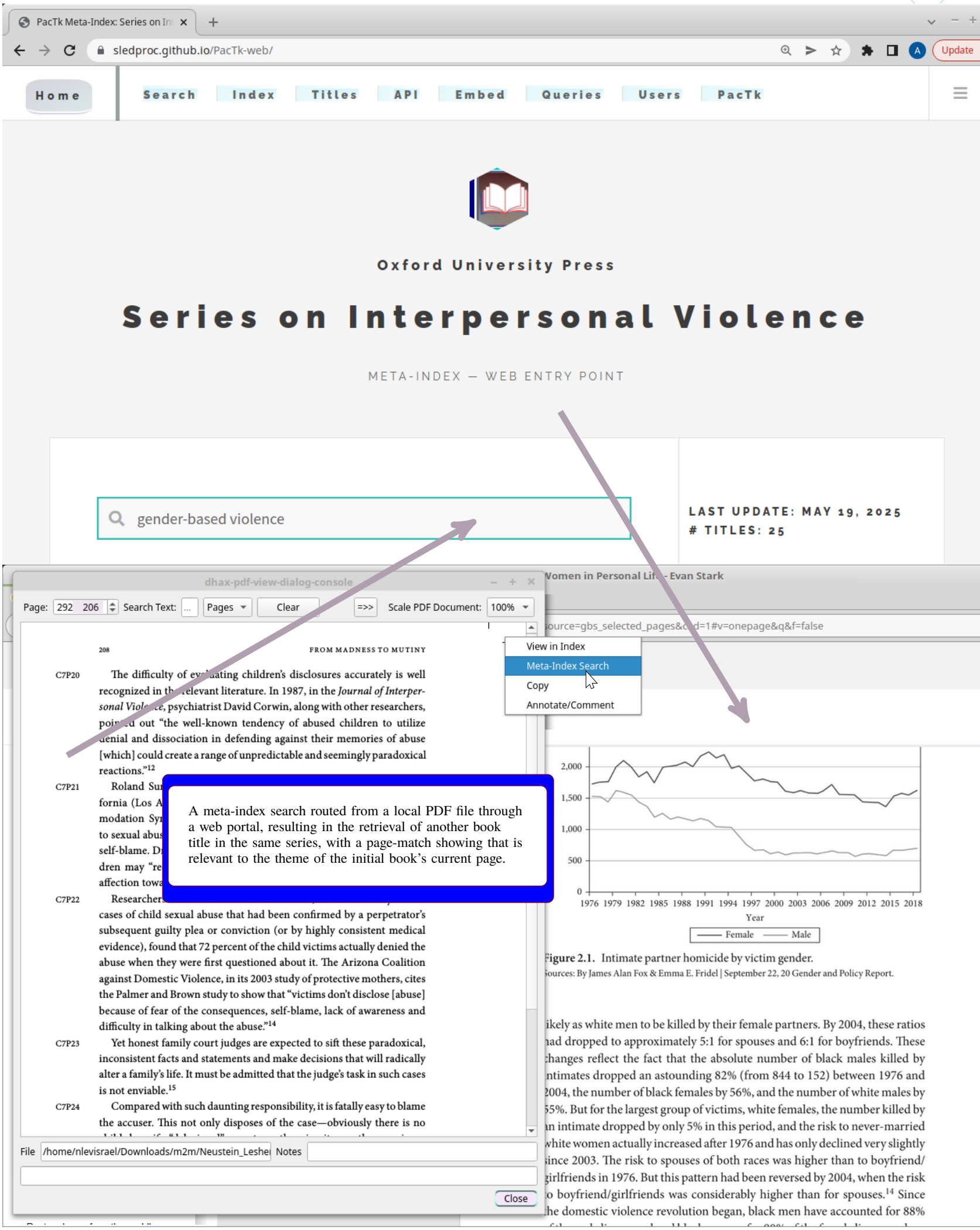


Figure 7: Meta-index search through a web portal

Some domain-specific file formats are derived from general-purpose specifications such as **CSV**, **XML**, and **JSON**. Others use idiosyncratic text and/or binary serialization protocols, so that custom parsers are necessary so as to consume byte or character streams from

relevant raw data files. Even when the actual encoding is performed via a generic standard like XML, dedicated deserialization code will still be needed — generic parsers may convert a given XML document to a DOM infoset, but algorithms will have to navigate the document hierarchy and extract text strings from element nodes, which become the input for initializing object data fields.

PacTk seeks to address these issues by offering a platform-independent Virtual Machine that is easy to embed in scientific applications or build as a standalone program. Scripting or query language can then emit code for this specific VM to take responsibility for deserializing encoded data sets. Depending on format, VM instructions might navigate through graph-like structures in a standardized profile (like XML) or implement and/or integrate with parsers for custom input file types, via postprocessors or rule-match callbacks.

Via these technologies, publishers can ensure that data sets are accessible so long as there are deserialization libraries targeting a PacTk VM, rather than duplicating effort (requiring multiple libraries for different operating systems and programming environments).

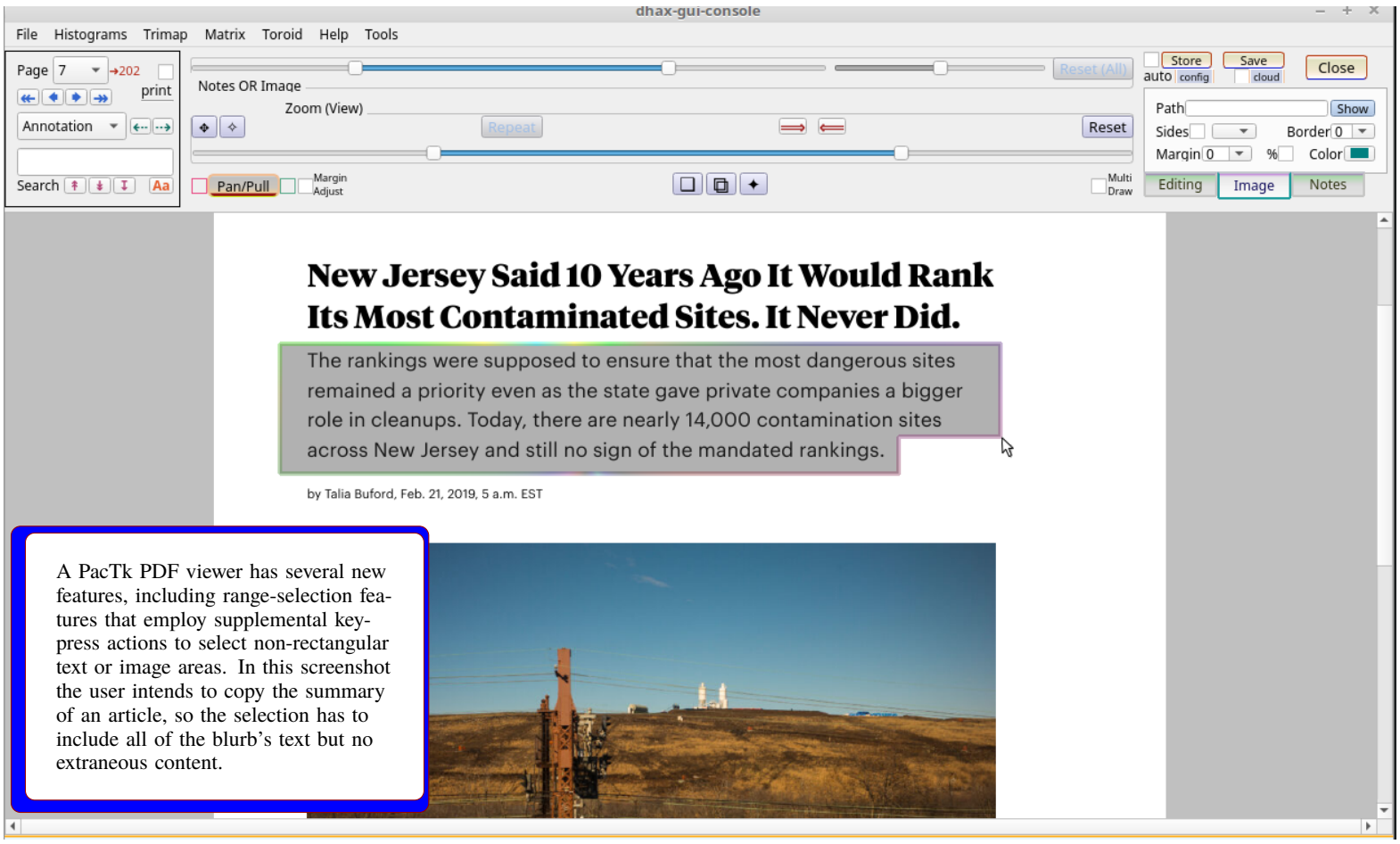


Figure 8: Flexible range-select features

Analytic and Demonstrative Code Many data sets are published alongside copies or samples of code used to analyze the raw data. Code sharing is important for transparency (allowing outsiders to double-check the accuracy of calculations and conclusion drawn therefrom) and reuse (replication of experiments/research projects). Data publications (bundled as Research Objects or similar scientific packages) may also include implementations for plugins targeted at applications charged with loading the raw data files. PacTk's object system includes several classes encapsulating plugin functionality (such as dynamic-library symbol resolution) and components exposed by host applications to extension modules (main-window taskbars, top/context menus, filesystem access, etc.).

Although such accompanying code may be implemented in many different languages and environments, the VM principles identified above for deserialization tasks also apply to code specific to research data sets. Here, too, accessibility implies that dataset code should be available in a cross-platform manner. Moreover, implementations should proceed via conventions suitable for GUI wrappers and interop, because research methods will often depend on domain-specific scientific or academic software. Ideally, data sets will be examined via plugins specific to applications familiar to the relevant research community, or at least custom-built software that can interoperate with domain-specific technologies. Unfortunately, many of the most popular scripting formats in scientific/research contexts (e.g., Python, JavaScript, and R) have noticeable limitations vis-à-vis native-GUI integration.

PacTk enables a different approach where languages can be customized to individual data sets and target pure-C++ VMs whose entire parsing, IR, and runtime stack can be embedded (as source code) in C++ GUI applications with no outside dependencies. Compilers and runtime engines for dataset code may thereby be released as part of the publication package itself. The VM and peer components could also be built directly into PDF viewers, multimedia applications (for videos, digital maps, etc.) and scientific computing platforms.

Another PacTk benefit for dataset code involves documentation and pedagogical features. In these contexts, code serves not merely to derive specific computations but also to demonstrate the acquisition methods and analytic protocols relevant for a research project. Such

details as units of measurement, expected ranges, typed coordinate systems (e.g., ensuring that procedures expecting horizontal-then-vertical coordinate pairs are never called with the converse), compile-time/statically-analyzable pre- and post-conditions, and so forth, help dataset code to serve as a resource clarifying scientific principles and theoretical models that drive published findings. The **PacTk** Virtual Machine includes overload-by-contract, dependent-typing (via a framework compatible with template metaprogramming), native interop (with dynamically loaded as well as statically compiled libraries) and "user-defined" calling-convention features that differ in some technical details from typical language interpreters, resulting in a scripting environment specifically built for open-access data sharing and the deployment of data packages as pedagogical tools.

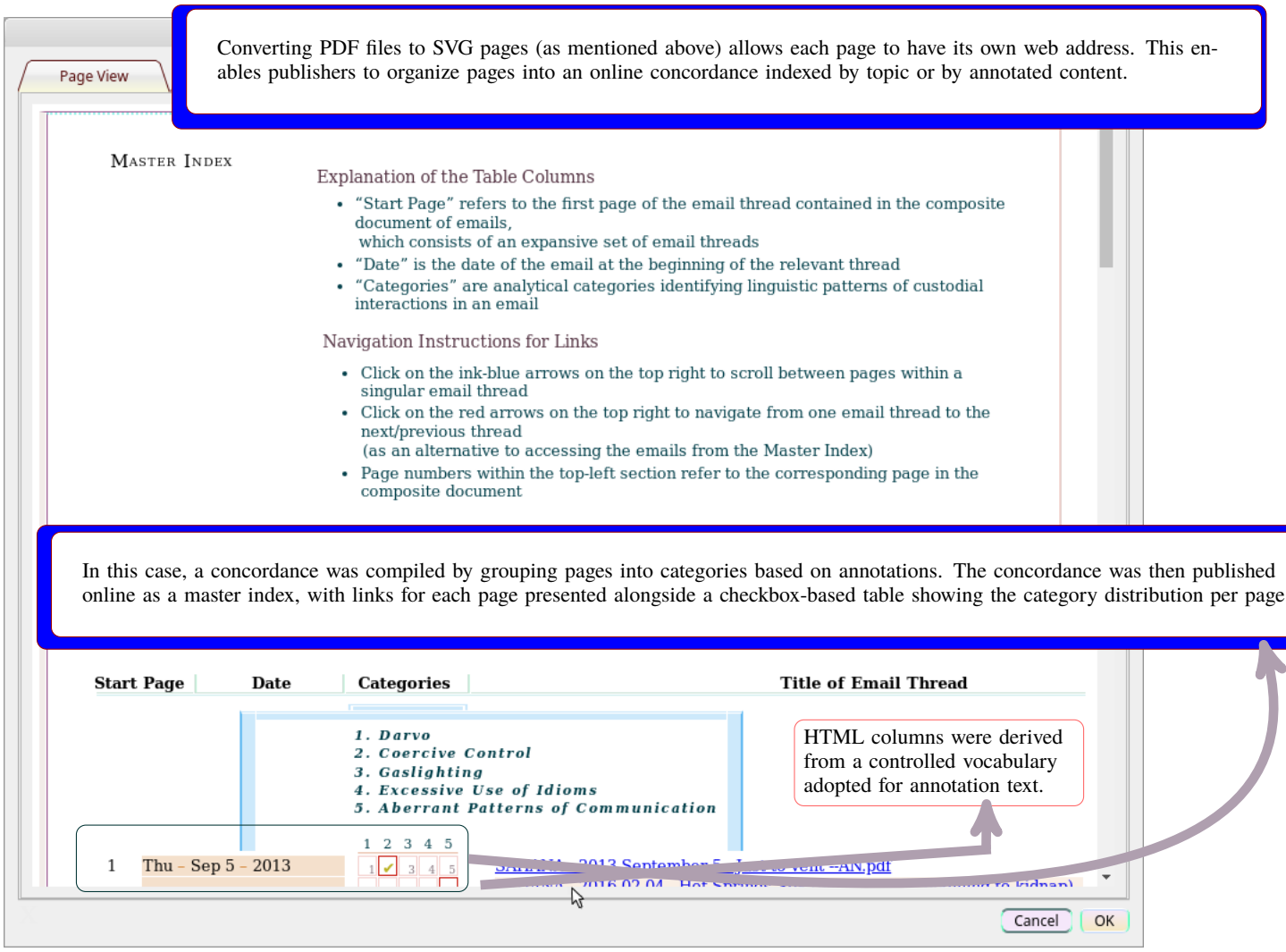


Figure 9: A category-based web-accessible index table for SVG pages

Full-Text Query Languages

As discussed earlier in this document, **PacTk** can be utilized as a code library by calling methods on **C++** (specifically, **TSOM**) objects. At the same time, a **PacTk** virtual machine supplies an environment for compiling scripting or query languages. These functionalities can be merged, such that the process of extracting information from a **PDF** manuscript, meta-indexed repository, or bibliographic database might be achieved by formulating and evaluating part of the code in a specialized query language. The techniques involved would be similar to employing **XQuery** or **XQSE** expressions to work with **JATS** or **BITS** files.

PacTk features its own flexible front-end parsing framework (mentioned above relative to **GTagML**) that could be used to program query grammars in an extensible, adaptable manner. By leveraging full regular expression capabilities, callbacks to arbitrary code (instead of generative production rules), and directly-executable grammar files (with no separate compilation steps), the **PacTk** parsers are more flexible than conventional **LEX/YACC**-style compilers. Meanwhile, the **VM** runtimes allows query evaluators to recognize basic scripting constructions such as locally-scoped variables and calls to arbitrary boolean-valued procedures as filter parameters.

In totality, such features allow text/bibliographic query systems to be implemented in a self-contained and cross-platform manner, with customizations specific to individual publishers and even to each particular journal or book series.

Multimedia Cross-Referencing

When a document (book or article) is explicitly paired with a published data set, information about the supplemental package should be included in index/search metadata. This would allow keyword searches to be extended to raw data fields, as well as data-structure parameters (such as column labels, source-code annotations, procedure names, etc.). Moreover, sections of text can be isolated as cross-references for data-structure parameters as well as individual objects/records. For example, a specific column or field in tabular data structures can be cross-referenced with the paragraph in article text where that field is discussed (its observation/acquisition methodology, units of measurement, valid range delineation, statistical distribution, etc.).

Cross-referencing between research papers and data sets is a special case of multimedia cross-referencing, where annotations defined in the context of a specific media type assert semantic or thematic relations to contents of a different modality. Concrete examples of multimedia cross-referencing include geotagging video frames; linking CAD digital twins with product specs; annotating Regions of Interest within image series with coded labels associated with published raw data; embedding tags in 360° displays whose content includes URL strings that encode identifiers for data set objects; developing color- and/or shape-coded attribute layers for GIS overlays (so users can browse between digital maps and structured object displays); and constructing numeric tuples from 3D graphics scenes such that particular camera orientation/location and zoom levels can be recorded as a unit, allowing the relevant 3D content to be repeatedly visualized from a specific perspective (in other words, links can be embedded in PDF files, web pages, software, or any other context so that users can reconstruct a specific 3D state, as constituted by camera angle/location and zoom settings).

The challenge when implementing multimedia cross-references is to ensure that viewers for two divergent media types may properly interoperate. With correctly interpreted cross-references, application users should be able to navigate from a window dedicated to one media/file to a window rendering a different format, and correctly isolate the portion of a file (not just the overall file) implicated in a cross-reference. For example, geotagging video frames should enable users to pause a video at a particular location and then, via the geotag, switch to a map display centered on the geospatial coordinates visible in the video (consider, e.g., a video documenting progress in an environmental remediation site). Conversely, GIS attribute layers would represent locations for which video content is available, and allow the users to watch the relevant videos starting from the appropriate time-point. Such interoperability is only possible if the components and/or applications responsible for rendering the two content-types share a common annotation and message-passing protocol. In the context of PacTk, this may be achieved via TSOM "message" objects that encapsulate remote-method requests and parameter serialization via TAGML-like structures.

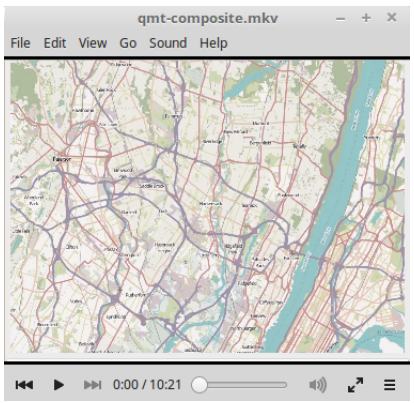
Science and Humanities GUIs Data publishing according to FAIRsharing standards often requires numerous project-specific resources in addition to raw data files. Fellow researchers typically need visualization and computational tools to fully leverage published data sets for evaluation and re-use. Unfortunately, relying on generic software to access published data packages is often suboptimal. For example, CSV files might be loaded into spreadsheets and/or statistical applications, but simply viewing data files in tabular fashion (even if the information is conducive to such structures) provides only limited value. The same is true for statistical plots and diagrams. Effective evaluation and re-use involves front-end programs that could provide both visual overviews (e.g., via charts and figures) and functionality for examining individual data objects in detail (e.g., via custom-designed GUI windows).

Furthermore, many data sets span multiple data profiles. For example, consider information tracking environmental pollutants, such as published by the Environmental Protection Agency's Toxic Release Inventory (TRI) project. Software tools for properly examining such data could supply both GIS maps (for understanding geospatial details and patterns about where environmental hazards are present) and chemistry-related software for molecular visualization, and other features that help document the biomedical effects, ecological consequences, and cleanup/remediation protocols for different classes of contaminants. Such components might further be extended by economic data profiling social impacts on surrounding communities, public health information, video and image series. etc.

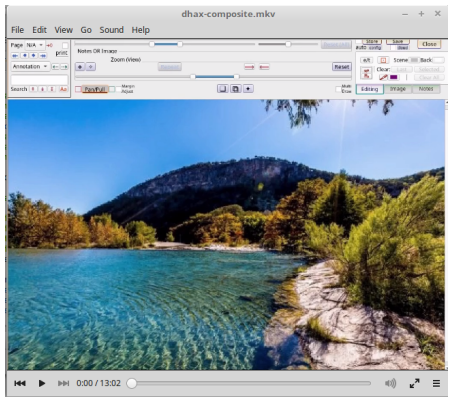
In general, then, publishers should not assume that FAIR transparency is achieved simply by releasing raw data files. Instead, open-access materials should in general include plugins or source-code extensions (developed individually for one manuscript or collectively for a book series, journal title, or document repository) targeting science applications with requisite capabilities to view/analyze domain-specific file types. Similar comments apply to humanities environments, such as front-ends for studying linguistics corpora and/or parse-structures; sociological/economic statistics; or graphics tools for digital humanities. PacTk can help in this context because a single PacTk plugin could serve as middleware on top of which plugins can run that are individualized to specific publications/manuscripts. This may be more efficient than building components against applications' native plugin mechanisms directly.

▷ Video Links

For more information, please see our software demo videos, which show software components that could be integrated into dataset applications and PDF viewers:



GIS



360° photography

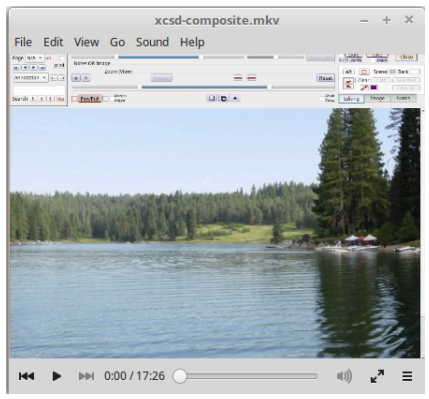


Image Processing